BloodDonationSystem - Authenticated

Generated with ZAP on Fri 26 Jun 2026, at 22:28:06

ZAP Version: 2.17.0

ZAP by Checkmarx

Contents

- About This Report
  - Report Description
  - Report Parameters
- Summaries
  - Alert Counts by Risk and Confidence
  - Alert Counts by Site and Risk
  - Alert Counts by Alert Type
- Alerts
  - Risk=High, Confidence=Medium (2)
  - Risk=High, Confidence=Low (2)
  - Risk=Medium, Confidence=High (2)
  - Risk=Medium, Confidence=Medium (4)

- Risk=Medium, Confidence=Low (1)
- Risk=Low, Confidence=High (1)
- Risk=Low, Confidence=Medium (5)
- Risk=Informational, Confidence=High (2)
- Risk=Informational, Confidence=Medium (5)
- Risk=Informational, Confidence=Low (1)

- Appendix
  - Alert Types

About This Report

Report Description

Authenticated: nurse01

Report Parameters

Contexts

No contexts were selected, so all contexts were included by default.

Sites

The following sites were included:

- https://cdn.jsdelivr.net
- http://192.168.100.50:5000

(If no sites were selected, all sites were included by default.)

An included site must also be within one of the included contexts for its data to be included in the report.

Risk levels

Included: High, Medium, Low, Informational

Excluded: None

Confidence levels

Included: User Confirmed, High, Medium, Low

Excluded: User Confirmed, High, Medium, Low, False Positive

Summaries

Alert Counts by Risk and Confidence

This table shows the number of alerts for each level of risk and confidence included in the report.

(The percentages in brackets represent the count as a percentage of the total number of alerts included in the report, rounded to one decimal place.)

		Confidence				
		User Confirmed	High	Medium	Low	Total
Risk	High	0 (0.0%)	0 (0.0%)	2 (8.0%)	2 (8.0%)	4 (16.0%)
	Medium	0 (0.0%)	2 (8.0%)	4 (16.0%)	1 (4.0%)	7 (28.0%)
	Low	0 (0.0%)	1 (4.0%)	5 (20.0%)	0 (0.0%)	6 (24.0%)
	Informational	0 (0.0%)	2 (8.0%)	5 (20.0%)	1 (4.0%)	8 (32.0%)
	Total	0 (0.0%)	5 (20.0%)	16 (64.0%)	4 (16.0%)	25 (100%)

	Confidence				
	User Confirmed	High	Medium	Low	Total

Alert Counts by Site and Risk

This table shows, for each site for which one or more alerts were raised, the number of alerts raised at each risk level.

Alerts with a confidence level of "False Positive" have been excluded from these counts.

(The numbers in brackets are the number of alerts raised for the site at or above that risk level.)

		Risk			
		High (= High)	Medium (>= Medium)	Low (>= Low)	Informational (>= Informational)
Site	https://cdn.jsdelivr.net	0 (0)	1 (1)	0 (1)	1 (2)
	http://192.168.100.50:5000	4 (4)	6 (10)	6 (16)	7 (23)

Alert Counts by Alert Type

This table shows the number of alerts of each alert type, together with the alert type's risk level.

(The percentages in brackets represent each count as a percentage, rounded to one decimal place, of the total number of alerts included in this report.)

Alert type	Risk	Count
Total		25

Alert type	Risk	Count
Cross Site Scripting (Persistent)	High	31 (124.0%)
Cross Site Scripting (Reflected)	High	10 (40.0%)
Path Traversal	High	5 (20.0%)
SQL Injection	High	5 (20.0%)
Absence of Anti-CSRF Tokens	Medium	3 (12.0%)
Buffer Overflow	Medium	32 (128.0%)
Content Security Policy (CSP) Header Not Set	Medium	3 (12.0%)
Cross-Domain Misconfiguration	Medium	2 (8.0%)
Format String Error	Medium	15 (60.0%)
Missing Anti-clickjacking Header	Medium	3 (12.0%)
Sub Resource Integrity Attribute Missing	Medium	5 (20.0%)
Application Error Disclosure	Low	7 (28.0%)
Cookie No HttpOnly Flag	Low	2 (8.0%)
Total		25

Alert type	Risk	Count
Cookie without SameSite Attribute	Low	2 (8.0%)
Cross-Domain JavaScript Source File Inclusion	Low	3 (12.0%)
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	5 (20.0%)
X-Content-Type-Options Header Missing	Low	4 (16.0%)
Authentication Request Identified	Informational	1 (4.0%)
GET for POST	Informational	1 (4.0%)
Information Disclosure - Suspicious Comments	Informational	13 (52.0%)
Modern Web Application	Informational	5 (20.0%)
Retrieved from Cache	Informational	2 (8.0%)
Session Management Response Identified	Informational	62 (248.0%)
User Agent Fuzzer	Informational	3 (12.0%)
User Controllable HTML Element Attribute (Potential XSS)	Informational	2 (8.0%)
Total		25

Alerts

Risk=High, Confidence=Medium (2)

http://192.168.100.50:5000 (2)

Cross Site Scripting (Persistent) (1)

▼ GET http://192.168.100.50:5000/staff/donor/2

Alert tags

- OWASP_2025_A05
- CWE-79
- OWASP_2021_A03
- PCI_DSS
- POLICY_DEV_FULL =
- POLICY_QA_STD =
- POLICY_QA_FULL =
- POLICY_PENTEST =
- HIPAA
- WSTG-v42-INPV-02
- OWASP_2017_A07

Alert description

Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology.

When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to

read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks essentially compromise the trust relationship between a user and the web site. Applications utilizing browser object instances which load content from the file system may execute code under the local machine zone allowing for system compromise.

There are three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-based.

Non-persistent attacks and DOM-based attacks require a user to either visit a specially crafted link laced with malicious code, or visit a malicious web page containing a web form, which when posted to the vulnerable site, will mount the attack. Using a malicious form will oftentimes take place when the vulnerable resource only accepts HTTP POST requests. In such a case, the form can be submitted automatically, without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious link or submitting the malicious form, the XSS payload will get echoed back and will get interpreted by the user's browser and execute. Another technique to send almost arbitrary requests (GET and POST) is by using an embedded client, such as Adobe Flash.

Persistent attacks occur when the malicious code is submitted to a web site where it's stored for a period of time. Examples of an attacker's favorite targets often include message board posts, web mail messages, and web chat software. The unsuspecting user is not required to interact with any additional site/link (e.g. an attacker site or a malicious link sent via email), just simply view the web page containing the code.

Other info

Source URL: http://192.168.100.50:5000/staff/donor/2/screening

Request

▼ Request line and header section (422 bytes)

```
GET
http://192.168.100.50:5000/staff/donor/2 HTTP/1.1
host: 192.168.100.50:5000
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
pragma: no-cache
cache-control: no-cache
referer:
http://192.168.100.50:5000/staff/donors
Cookie:
session=eyJyb2xlIjoibnVyc2UiLCJzdGFmZl9pZCI6MSwidXNlcm5hbWUiOiJudXJzZTAxIn0.aj6HQQ.voKYv02dmgUy1SXdIir48TwvsLY
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (190 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:05:53 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 74952
Vary: Cookie
Connection: close
```

▶ Response body (74952 bytes)

Parameter `disease`

Attack `</div><script>alert(1);</script><div>`

Solution

Phase: Architecture and Design

Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.

Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache Wicket.

Phases: Implementation; Architecture and Design

Understand the context in which your data will be used and the encoding that will be expected. This is especially important when transmitting data between different components, or when generating outputs that can contain multiple encodings at the same time,

such as web pages or multi-part mail messages. Study all expected communication protocols and data representations to determine the required encoding strategies.

For any data that will be output to another web page, especially any data that was received from external inputs, use the appropriate encoding on all non-alphanumeric characters.

Consult the XSS Prevention Cheat Sheet for more details on the types of encoding and escaping that are needed.

Phase: Architecture and Design

For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server.

If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated.

Phase: Implementation

For every web page that is generated, use and specify a character encoding such as ISO-8859-1 or UTF-8. When an encoding is not specified, the web browser may choose a different encoding by guessing which encoding is actually being used by the web page. This can cause the web browser to treat certain sequences as special, opening up the client to subtle XSS attacks. See CWE-116 for more mitigations related to encoding/escaping.

To help mitigate XSS attacks against the user's session cookie, set the session cookie to be HttpOnly. In browsers that support the HttpOnly feature (such as more recent versions of Internet Explorer and Firefox), this attribute can prevent the user's session cookie from being accessible to malicious client-side scripts that use document.cookie. This is not a complete solution, since HttpOnly is not supported by all browsers. More importantly, XMLHTTPRequest and other powerful browser technologies provide read access to HTTP headers, including the Set-Cookie header in which the HttpOnly flag is set.

Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or

determining which inputs are so malformed that they should be rejected outright.

When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."

Ensure that you perform input validation at well-defined interfaces within the application. This will help protect the application even if a component is reused or moved elsewhere.

Cross Site Scripting (Reflected) (1)

▼ POST http://192.168.100.50:5000/staff/donor/2/screening

Alert tags

- CWE-79
- POLICY_SEQUENCE =
- OWASP_2021_A03
- PCI_DSS
- WSTG-v42-INPV-01
- POLICY_QA_CICD =
- POLICY_DEV_CICD =
- OWASP_2025_A05
- POLICY_DEV_FULL =
- POLICY_QA_STD =
- POLICY_QA_FULL =
- POLICY_PENTEST =
- HIPAA
- OWASP_2017_A07

- POLICY_DEV_STD =

Alert description

Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology.

When an attacker gets a user's browser to execute his/her code, the code will run within the security context (or zone) of the hosting web site. With this level of privilege, the code has the ability to read, modify and transmit any sensitive data accessible by the browser. A Cross-site Scripted user could have his/her account hijacked (cookie theft), their browser redirected to another location, or possibly shown fraudulent content delivered by the web site they are visiting. Cross-site Scripting attacks essentially compromise the trust relationship between a user and the web site. Applications utilizing browser object instances which load content from the file system may execute code under the local machine zone allowing for system compromise.

There are three types of Cross-site Scripting attacks: non-persistent, persistent and DOM-based.

Non-persistent attacks and DOM-based attacks require a user to either visit a

specially crafted link laced with malicious code, or visit a malicious web page containing a web form, which when posted to the vulnerable site, will mount the attack. Using a malicious form will oftentimes take place when the vulnerable resource only accepts HTTP POST requests. In such a case, the form can be submitted automatically, without the victim's knowledge (e.g. by using JavaScript). Upon clicking on the malicious link or submitting the malicious form, the XSS payload will get echoed back and will get interpreted by the user's browser and execute. Another technique to send almost arbitrary requests (GET and POST) is by using an embedded client, such as Adobe Flash.

Persistent attacks occur when the malicious code is submitted to a web site where it's stored for a period of time. Examples of an attacker's favorite targets often include message board posts, web mail messages, and web chat software. The unsuspecting user is not required to interact with any additional site/link (e.g. an attacker site or a malicious link sent via email), just simply view the web page containing the code.

Request

▼ Request line and header section (561 bytes)

```
POST
http://192.168.100.50:5000/staff/donor/2/screening HTTP/1.1
host: 192.168.100.50:5000
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0
```

```
Safari/537.36
pragma: no-cache
cache-control: no-cache
content-type: application/x-www-form-
urlencoded
referer:
http://192.168.100.50:5000/staff/dono
r/2
content-length: 126
Cookie:
session=.eJw9ikEKgCAUBa8iby2RW8_QDUJE
7FuCKfh1Fd49F9FqGGYe2JAcX8TQ-
wPRJsDde2KGxFbOmMXnoacFZhiJWhJBI_fKNC
9uLgQbD2gl0ZlqdvffV4XxAu2nIi4.aj6HLQ.
6CbEj4VJJFDJzvQAkyeT4_oll7A
```

▼ Request body (126 bytes)

```
blood_type=A-
&weight=1&height=1&disease=%3C%2Fdiv%
3E%3CscrIpt%3Ealert%281%29%3B%3C%2Fsc
Ript%3E%3Cdiv%3E&result=Failed&remark
s=
```

Response

▼ Status line and header section (322 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:05:33 GMT
Content-Type: text/html; charset=utf-
8
Content-Length: 67073
Vary: Cookie
Set-Cookie:
session=eyJyb2xlIjoibnVyc2UiLCJzdGFmZ
l9pZCI6MSwidXNlcm5hbWUiOiJudXJzZTAxIn
0.aj6HLQ.F0KSRcJDj0CF7g0rDtZIISwM98A;
 Path=/
Connection: close
```

▶ Response body (67073 bytes)

Parameter	`disease`
Attack	`</div><scrIpt>alert(1);</scRipt><div>`
Evidence	`</div><scrIpt>alert(1);</scRipt><div>`
Solution	Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. Examples of libraries and frameworks that make it easier to generate properly encoded output include Microsoft's Anti-XSS library, the OWASP ESAPI Encoding module, and Apache Wicket. Phases: Implementation; Architecture and Design Understand the context in which your data will be used and the encoding that will be expected. This is especially important when transmitting data between different components, or when generating outputs that can contain multiple encodings at the same time, such as web pages or multi-part mail messages. Study all expected communication protocols and data representations to determine the required encoding strategies. For any data that will be output to another web page, especially any data that was received from external inputs, use the appropriate encoding on all non-alphanumeric characters.

Consult the XSS Prevention Cheat Sheet for more details on the types of encoding and escaping that are needed.

Phase: Architecture and Design

For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server.

If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated.

Phase: Implementation

For every web page that is generated, use and specify a character encoding such as ISO-8859-1 or UTF-8. When an encoding is not specified, the web browser may choose a different encoding by guessing which encoding is actually being used by the web page. This can cause the web browser to treat certain sequences as special, opening up the client to subtle XSS attacks. See CWE-116 for more mitigations related to encoding/escaping.

To help mitigate XSS attacks against the user's session cookie, set the session cookie to be HttpOnly. In browsers that support the HttpOnly feature (such as more recent versions of Internet Explorer and Firefox), this attribute can prevent the user's session cookie from being accessible to malicious client-side scripts that use document.cookie. This is not a complete solution, since HttpOnly is not supported by all browsers. More importantly, XMLHTTPRequest and other powerful browser technologies provide read access to HTTP headers, including the Set-Cookie header in which the HttpOnly flag is set.

Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright.

When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it

only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."

Ensure that you perform input validation at well-defined interfaces within the application. This will help protect the application even if a component is reused or moved elsewhere.

Risk=High, Confidence=Low (2)

http://192.168.100.50:5000 (2)

Path Traversal (1)

▼ POST http://192.168.100.50:5000/staff/donor/1/screening

Alert tags

- OWASP_2021_A01
- POLICY_SEQUENCE =
- CWE-22
- PCI_DSS
- OWASP_2025_A01
- WSTG-v42-ATHZ-01
- POLICY_DEV_FULL =
- POLICY_QA_STD =
- POLICY_QA_FULL =
- POLICY_PENTEST =
- HIPAA
- OWASP_2017_A05
- POLICY_DEV_STD =

Alert description

The Path Traversal attack technique allows an attacker access to files, directories, and commands that potentially reside outside the web document root directory. An attacker may manipulate a URL in such a way that the web site will execute or reveal

the contents of arbitrary files anywhere on the web server. Any device that exposes an HTTP-based interface is potentially vulnerable to Path Traversal.

Most web sites restrict user access to a specific portion of the file-system, typically called the "web document root" or "CGI root" directory. These directories contain the files intended for user access and the executable necessary to drive web application functionality. To access files or execute commands anywhere on the file-system, Path Traversal attacks will utilize the ability of special-characters sequences.

The most basic Path Traversal attack uses the "../" special-character sequence to alter the resource location requested in the URL. Although most popular web servers will prevent this technique from escaping the web document root, alternate encodings of the "../" sequence may help bypass the security filters. These method variations include valid and invalid Unicode-encoding ("..%u2216" or "..%c0%af") of the forward slash character, backslash characters ("..\") on Windows-based servers, URL encoded characters "%2e%2e%2f"), and double URL encoding ("..%255c") of the backslash character.

Even if the web server properly restricts Path Traversal attempts in the URL path, a web application itself may still be vulnerable due to improper handling of user-supplied input. This is a common problem of web applications that use template mechanisms or load static text

from files. In variations of the attack, the original URL parameter value is substituted with the file name of one of the web application's dynamic scripts. Consequently, the results can reveal source code because the file is interpreted as text instead of an executable script. These techniques often employ additional special characters such as the dot (".") to reveal the listing of the current working directory, or "%00" NULL characters in order to bypass rudimentary file extension checks.

Request

▼ Request line and header section (560 bytes)

```
POST
http://192.168.100.50:5000/staff/donor/1/screening HTTP/1.1
host: 192.168.100.50:5000
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
pragma: no-cache
cache-control: no-cache
content-type: application/x-www-form-urlencoded
referer:
http://192.168.100.50:5000/staff/donor/1
content-length: 69
Cookie:
session=.eJw9ikEKgCAUBa8iby2RW8_QDUJE7FuCKfh1Fd49F9FqGGYe2JAcX8TQ-wPRJsDde2KGxFbOmMXnoacFZhiJWhJBI_fKNC9uLgQbD2gl0ZlqdvffV4XxAu2nIi4.aj6G1A.jQrnKuflrsBGR8WbIwC9fuBb4oA
```

▼ Request body (69 bytes)

blood_type=A-&weight=1&height=1&disease=ZAP&result=screening&remarks=

Response

▼ Status line and header section (322 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:04:04 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 21312
Vary: Cookie
Set-Cookie: session=eyJyb2xlIjoibnVyc2UiLCJzdGFmZl9pZCI6MSwidXNlcm5hbWUiOiJudXJzZTAxIn0.aj6G1A.REQzp1SFQs8ojDrKgDVexEL335M; Path=/
Connection: close
```

▶ Response body (21312 bytes)

Parameter result

Attack screening

Solution Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use an allow list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. Do not rely exclusively on looking for malicious or malformed inputs (i.e., do not rely on a deny list). However, deny lists can be useful for detecting potential attacks or determining which inputs are so

malformed that they should be rejected outright.

When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if you are expecting colors such as "red" or "blue."

For filenames, use stringent allow lists that limit the character set to be used. If feasible, only allow a single "." character in the filename to avoid weaknesses, and exclude directory separators such as "/". Use an allow list of allowable file extensions.

Warning: if you attempt to cleanse your data, then do so that the end result is not in the form that can be dangerous. A sanitizing mechanism can remove characters such as '.' and ';' which may be required for some exploits. An attacker can try to fool the sanitizing mechanism into "cleaning" data into a dangerous form. Suppose the attacker injects a '.' inside a filename (e.g. "sensi.tiveFile") and the sanitizing mechanism removes the character resulting in the valid filename, "sensitiveFile". If the input data are now assumed to be safe, then the file may be compromised.

Inputs should be decoded and canonicalized to the application's

current internal representation before being validated. Make sure that your application does not decode the same input twice. Such errors could be used to bypass allow list schemes by introducing dangerous inputs after they have been checked.

Use a built-in path canonicalization function (such as realpath() in C) that produces the canonical version of the pathname, which effectively removes ".." sequences and symbolic links.

Run your code using the lowest privileges that are required to accomplish the necessary tasks. If possible, create isolated accounts with limited privileges that are only used for a single task. That way, a successful attack will not immediately give the attacker access to the rest of the software or its environment. For example, database applications rarely need to run as the database administrator, especially in day-to-day operations.

When the set of acceptable objects, such as filenames or URLs, is limited or known, create a mapping from a set of fixed input values (such as numeric IDs) to the actual filenames or URLs, and reject all other inputs.

Run your code in a "jail" or similar sandbox environment that enforces strict boundaries between the process and the operating system. This may effectively restrict which files can be accessed in a particular directory or which commands can be executed by your software.

OS-level examples include the Unix chroot jail, AppArmor, and SELinux. In general, managed code may provide some protection. For example, java.io.FilePermission in the Java SecurityManager allows you to specify restrictions on file operations.

This may not be a feasible solution, and it only limits the impact to the operating system; the rest of your application may still be subject to compromise.

SQL Injection (1)

▼ POST http://192.168.100.50:5000/staff/login

Alert tags

- POLICY_SEQUENCE =
- OWASP_2021_A03
- PCI_DSS
- CWE-89
- POLICY_QA_CICD =
- POLICY_DEV_CICD =
- OWASP_2025_A05
- WSTG-v42-INPV-05
- POLICY_API =
- POLICY_DEV_FULL =
- POLICY_QA_STD =
- POLICY_QA_FULL =
- POLICY_PENTEST =
- HIPAA
- OWASP_2017_A01
- API_2023_API10
- POLICY_DEV_STD =

Alert description

SQL injection may be possible.

Request

▼ Request line and header section (546 bytes)

```
POST
http://192.168.100.50:5000/staff/log
in HTTP/1.1
host: 192.168.100.50:5000
user-agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/141.0.0.0
Safari/537.36
pragma: no-cache
cache-control: no-cache
content-type: application/x-www-
form-urlencoded
referer:
http://192.168.100.50:5000/staff/log
in
content-length: 25
Cookie:
session=.eJw9ikEKgCAUBa8iby2RW8_QDUJ
E7FuCKfh1Fd49F9FqGGYe2JAcX8TQ-
wPRJsDde2KGxFbOmMXnoacFZhiJWhJBI_fKN
C9uLgQbD2gl0ZlqdvffV4XxAu2nIi4.aj6HY
A.6wVVdOOrqdEH8HU6OO968vqJ9Uc
```

▼ Request body (25 bytes)

```
username=%27&password=ZAP
```

Response

▼ Status line and header section (340 bytes)

```
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:06:24 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 2213
Vary: Cookie
Set-Cookie:
session=eyJyb2xlIjoibnVyc2UiLCJzdGFm
Zl9pZCI6MSwidXNlcm5hbWUiOiJudXJzZTAx
In0.aj6HYA.3KrL_VhiDiezDKMxFGD3xrjs7
```

```
GQ; Path=/
Connection: close
```

▶ Response body (2213 bytes)

Parameter username

Attack '

Evidence HTTP/1.1 500 INTERNAL SERVER ERROR

Solution

Do not trust client side input, even if there is client side validation in place.

In general, type check all data on the server side.

If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?'

If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries.

If database Stored Procedures can be used, use them.

Do *not* concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality!

Do not create dynamic SQL queries using simple string concatenation.

Escape all data received from the client.

Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input.

Apply the principle of least privilege by using the least privileged database user possible.

In particular, avoid using the 'sa' or 'db-owner' database users. This does not eliminate SQL injection, but minimizes its impact.

Grant the minimum database access that is necessary for the application.

Risk=Medium, Confidence=High (2)

http://192.168.100.50:5000 (2)

Content Security Policy (CSP) Header Not Set (1)

▼ GET http://192.168.100.50:5000/

Alert tags

- OWASP_2021_A05
- POLICY_QA_STD =
- POLICY_PENTEST =
- SYSTEMIC
- CWE-693
- OWASP_2017_A06
- OWASP_2025_A02

Alert description

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a

set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.

Request

▼ Request line and header section (333 bytes)

```
GET http://192.168.100.50:5000/
HTTP/1.1
host: 192.168.100.50:5000
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept:
text/html,application/xhtml+xml,appli
cation/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (189 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:02:14 GMT
Content-Type: text/html; charset=utf-
8
Content-Length: 2563
Vary: Cookie
Connection: close
```

▶ Response body (2563 bytes)

Solution

Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.

Sub Resource Integrity Attribute Missing (1)

▼ GET http://192.168.100.50:5000/

Alert tags

- CWE-345
- OWASP_2021_A05
- POLICY_QA_STD =
- POLICY_PENTEST =
- SYSTEMIC
- OWASP_2017_A06
- OWASP_2025_A02
- POLICY_DEV_STD =

Alert description

The integrity attribute is missing on a script or link tag served by an external server. The integrity tag prevents an attacker who have gained access to this server from injecting a malicious content.

Request

▼ Request line and header section (333 bytes)

```
GET http://192.168.100.50:5000/
HTTP/1.1
host: 192.168.100.50:5000
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept:
text/html,application/xhtml+xml,appl
ication/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
```

```
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (189 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:02:14 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 2563
Vary: Cookie
Connection: close
```

▶ Response body (2563 bytes)

Evidence

```
<link
href="https://cdn.jsdelivr.net/npm/b
ootstrap@5.3.3/dist/css/bootstrap.mi
n.css" rel="stylesheet">
```

Solution

Provide a valid integrity attribute to the tag.

Risk=Medium, Confidence=Medium (4)

https://cdn.jsdelivr.net (1)

Cross-Domain Misconfiguration (1)

▼ GET
https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css

Alert tags

- OWASP_2021_A01
- POLICY_QA_STD =
- POLICY_PENTEST =
- SYSTEMIC
- OWASP_2025_A01
- OWASP_2017_A05
- CWE-264

Alert description

Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web server.

Other info

The CORS misconfiguration on the web server permits cross-domain read requests from arbitrary third party domains, using unauthenticated APIs on this domain. Web browser implementations do not permit arbitrary third parties to read the response from authenticated APIs, however. This reduces the risk somewhat. This misconfiguration could be used by an attacker to access data that is available in an unauthenticated manner, but which uses some other form of security, such as IP address white-listing.

Request

▼ Request line and header section (410 bytes)

```
GET
https://cdn.jsdelivr.net/npm/bootstra
p@5.3.3/dist/css/bootstrap.min.css
HTTP/1.1
host: cdn.jsdelivr.net
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/css,*/*;q=0.1
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
```

```
Referer: http://192.168.100.50:5000/
Sec-Fetch-Dest: style
Sec-Fetch-Mode: no-cors
Sec-Fetch-Site: cross-site
Priority: u=2
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (768 bytes)

```
HTTP/1.1 200 OK
Connection: keep-alive
Content-Length: 232803
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: *
Timing-Allow-Origin: *
Cache-Control: public, max-
age=31536000, s-maxage=31536000,
immutable
Cross-Origin-Resource-Policy: cross-
origin
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-
age=31536000; includeSubDomains;
preload
Content-Type: text/css; charset=utf-8
X-JSD-Version: 5.3.3
X-JSD-Version-Type: version
ETag: W/"38d63-
xawd7pYctZoEUlbsID9p4xeHL3w"
Accept-Ranges: bytes
Age: 2703802
Date: Fri, 26 Jun 2026 14:02:15 GMT
X-Served-By: cache-fra-etou8220150-
FRA, cache-sin-wsss1830094-SIN
X-Cache: HIT, HIT
Vary: Accept-Encoding
alt-svc: h3=":443";ma=86400,h3-
29=":443";ma=86400,h3-
```

27=":443";ma=86400

▶ Response body (232803 bytes)

Evidence `Access-Control-Allow-Origin: *`

Solution Ensure that sensitive data is not available in an unauthenticated manner (using IP address white-listing, for instance).

Configure the "Access-Control-Allow-Origin" HTTP header to a more restrictive set of domains, or remove all CORS headers entirely, to allow the web browser to enforce the Same Origin Policy (SOP) in a more restrictive manner.

http://192.168.100.50:5000 (3)

Buffer Overflow (1)

▼ POST http://192.168.100.50:5000/staff/donor/11/screening

Alert tags

- OWASP_2025_A05
- OWASP_2021_A03
- PCI_DSS
- POLICY_API =
- POLICY_PENTEST =
- OWASP_2017_A01
- API_2023_API10
- OWASP_2025_A10
- CWE-120

Alert description Buffer overflow errors are characterized by the overwriting of memory spaces of the background web process, which

should have never been modified intentionally or unintentionally. Overwriting values of the IP (Instruction Pointer), BP (Base Pointer) and other registers causes exceptions, segmentation faults, and other process errors to occur. Usually these errors end execution of the application in an unexpected way.

Other info

Potential Buffer Overflow. The script closed the connection and threw a 500 Internal Server Error.

Request

▼ Request line and header section (569 bytes)

```
POST
http://192.168.100.50:5000/staff/donor/11/screening HTTP/1.1
host: 192.168.100.50:5000
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
pragma: no-cache
cache-control: no-cache
content-type: application/x-www-form-urlencoded
referer:
http://192.168.100.50:5000/staff/donor/11
content-length: 2164
Cookie:
session=.eJzNikEKgCAUBa8iby2RW8_QDUJC7FuCKfhzJd49F9EZWg3DTMPmo-WTGHptEPcAuDpHzJBY8hGSeN3XOMF0-ePNSJQcCRqpFqZx8W2938IOrSQqU0n2-vqs0B_5ela2.aj6LSA.fnyNLVrR1hIdbGN5vjx-dRgGqNU
```

▶ Request body (2164 bytes)

Response

▼ Status line and header section (195 bytes)

```
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:23:04 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 16726
Connection: close
```

▶ Response body (16726 bytes)

Parameter blood_type

Attack

CrmbJXhrjeTHQohOldqcOXbRoLuKcDtvonBSl
BaBraPbWhqdEXybNRTEWShMyvWGeiEucmdQVq
bRHrhruWgRdutEnAwZidXwjKRbatTRnyaaJdJ
FKDydNHZOTnvjYYsLFiTdDKHxOMFmpAPFDXsE
TdxXbqmUhxplSiCmnxjkkIWrmefJtgQXjBklg
JFulWFpWsMsaxFjitcpNHjdMeGKqoWOkjbXCk
RByIZlJFySmEODxHUVtVcqIcaUtuwLywGdmlO
jypkiNoMtxSKfOTFlHKaRfqxftdCfEExpVJeX
SImaWghtJEFEFmQErUvqaxMCdXSiAZBQtVIsj
AYgnxOmsorQCIGxJqocIOBQFclUJfURAaEBHR
QITKeGHyaRhqRQkSkXncGanRXCggXpbkUIUul
uhMvLUsmIiOxRBPQUMMnpPWiaRHoSttlfgnGN
XwiqIuiOkhjpCoRlwNIRscmcqsSCDfGmGcYwq
KrZyTumPvHndpIFCBmmBoObvgwdrIsfkltSoK
MmXaGZeVBwkTGapAydgdbNoVgKaIqadjjtyBt
mMaRAMbwlEDeCrsqaPKGawTQKApnviatKmJId
YxSgrvVnibnPkXAJGSbExxPFrSbtJhIdcMGpv
eAkYCwidQmYJCZEkbwxBygrHhssYGvLGyqtlS
FDjIPVAVenyyVngelTTYDlawBWQIajdnvRdrb
cBwHMXowIgSNmBEMiWCDACBugUZnGhbPjKXKN
SqpTBLhpBsfNWGASjNWaPMktQFTtqBJxIPySS
eNsVLXrwhAftaHkoOZgLLGEiLIjVIBQpiCIAV
qnnKyxojcNwZfJOCXYJVlioRyXQDXIHPGdyxd
EEIDGOVxIMgRFeMmVmfRPxifBDLOhXgCMoXdc
nGfhgOGUPWjIpmTuoWkQafADQFnOsXcaxNUKg

CVRbYIUVQahwQtuTxwnsbiZjZQvetgbwaJjNV
kSTPlLvipuEsFMtsKoNjMHBIWUsivfWVCUNuT
yLoJqTiTaodrSnEsXdHTcmRoPvMeuxWkNcGjD
mtDBEeAGkiRKCCGNWvimgvDiZQWpaLwhAAmvo
laEapdsMlBnvRlNGmOtubmdYRdJNvNPDkUtCw
ZONfotMASiaGBsNNtQvslJSfQkVOuynQLLBmG
tccuhsMYZaFfAbKtWNaMKEHIoDWStEdbZvdOA
QPICIpAnNVynoExCVxswgaibToSFfvWJRolsF
WCCZvkYpDxvnDXeEuTuFZqpUdOcctfeEJyPIC
nOoOSxCIceiZtFDsyNeqZUPJhovqtpowlkKkL
AMuZjblgqBsEbiJeqiBBcYxCSTDyyDIbeIZpo
ASjTiIDtHTsEuuGtcoJDRmWifaScjUhGwxkpF
jpySECCNBaRKElxmKCdRNeBASpKhCulFTQoZV
qREjKOumfuDtQqEGWlNeOXjqWFqgxroiwHdUx
XIULNtdFaBuNQgQBQdKwuUblbpAWCyrlqZonH
GirZoiJHYgxFMVoVXYapyAdvFoGPYBPcLHFEZ
begbmRNLxdWPdwKTfTgjlamGuTSokhoybmVYA
HnoPLmdmbJYHFaBThEtLGpwYsyIUQURQxoAHB
tpXMPsVhmlHObrUSPXxsiYcyUjJhCkwMiiqeb
lDLatjAbcRIdyGVMTqiPsOiyxvbwmhcfYSAsc
IdsXJklEPQNUNlYuloTtBCboaKRJqAmfdYAXL
TqiZhTPWVcfeiwcLFVxLBQmVHMxjDPGLgkYXO
YcsjpMEbKdGTOrGRiXdroIDcqgUTWnZAIejvE
HHJqnBnVYCEBcdpiEJhatVfOMaCGXrZdgHtXf
uOjaCBeZLNlDnmjiWhAhjOZaqXowAsthHflWD
GSxAHEJqUtZjpMCPBbyWOZEnhlXBwUJePqsID
NoDKAdAqkVUyaULjoNcaJYBRXkdRAVwgKetsl
INAkvHFFUUWZCoUlledmHwlTtmoIgHZVraJui
mshwtaHedreHmsVolqHZFIkebdHgWfIggLMKD
ApLQamXKtXInPdAQkaYiPadndUTiXmKAZjgxB
ZlqpLmHrFOFjmPhGAcXWTPEwCRCtPtFOaSOnj
kpuKiDdRgXhrVWEMplSklmTpPDpK

Evidence `Connection: close`

Solution Rewrite the background program using proper return length checking. This will require a recompile of the background executable.

Format String Error (1)

▼ POST http://192.168.100.50:5000/staff/donor/2/screening

Alert tags

- OWASP_2025_A05
- OWASP_2021_A03
- POLICY_API =
- POLICY_QA_FULL =
- POLICY_PENTEST =
- OWASP_2017_A01
- API_2023_API10
- OWASP_2025_A10
- CWE-134

Alert description

A Format String error occurs when the submitted data of an input string is evaluated as a command by the application.

Other info

Potential Format String Error. The script closed the connection on a /%s.

Request

▼ Request line and header section (566 bytes)

```
POST
http://192.168.100.50:5000/staff/don
or/2/screening HTTP/1.1
host: 192.168.100.50:5000
user-agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/141.0.0.0
Safari/537.36
pragma: no-cache
cache-control: no-cache
content-type: application/x-www-
form-urlencoded
referer:
http://192.168.100.50:5000/staff/don
or/2
content-length: 230
Cookie:
session=.eJyNikEKgCAUBa8iby2RW8_QDUJ
C7FuCKfhzJd49F9G61TDMNGw-Wj6JodcGcQ-
```

Aq3PEDIklHyGJ132NE0yX_zYjUXIkaKRamMb
Ft_V-
Czu0kqhMJdnr67NCfwAwNS9Q.aj6LUA.91Ej
DukFoCLW0tyfFfrWZaROUg8

▼ Request body (230 bytes)

```
blood_type=ZAP%25n%25s%25n%25s%25n%2
5s%25n%25s%25n%25s%25n%25s%25n%25s%2
5n%25s%25n%25s%25n%25s%25n%25s%25n%2
5s%25n%25s%25n%25s%25n%25s%25n%25s%2
5n%25s%25n%25s%25n%25s%25n%25s%0A&we
ight=1&height=1&disease=ZAP&result=F
ailed&remarks=
```

Response ▼ Status line and header section (195 bytes)

```
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:23:13 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 16726
Connection: close
```

▶ Response body (16726 bytes)

Parameter blood_type

Attack ZAP%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s%n%s

Solution Rewrite the background program using proper deletion of bad character strings. This will require a recompile of the background executable.

Missing Anti-clickjacking Header (1)

▼ GET http://192.168.100.50:5000/

Alert tags

- OWASP_2021_A05
- POLICY_QA_STD =
- POLICY_PENTEST =
- CWE-1021
- SYSTEMIC
- WSTG-v42-CLNT-09
- OWASP_2017_A06
- OWASP_2025_A02

Alert description

The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

Request

▼ Request line and header section (333 bytes)

```
GET http://192.168.100.50:5000/
HTTP/1.1
host: 192.168.100.50:5000
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept:
text/html,application/xhtml+xml,appl
ication/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (189 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:02:14 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 2563
Vary: Cookie
Connection: close
```

▶ Response body (2563 bytes)

Parameter `x-frame-options`

Solution Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app.

If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.

Risk=Medium, Confidence=Low (1)

http://192.168.100.50:5000 (1)

Absence of Anti-CSRF Tokens (1)

▼ GET http://192.168.100.50:5000/staff/login

Alert tags
- OWASP_2021_A01
- POLICY_QA_STD =

- POLICY_PENTEST =
- SYSTEMIC
- WSTG-v42-SESS-05
- OWASP_2025_A01
- OWASP_2017_A05
- CWE-352
- POLICY_DEV_STD =

Alert description

No Anti-CSRF tokens were found in a HTML submission form.

A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf.

CSRF attacks are effective in a number of situations, including:

* The victim has an active session on the target site.

* The victim is authenticated via HTTP auth on the target site.

* The victim is on the same local network as the target site.

CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of the same-origin policy.

Other info

No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "password" "username"].

Request

▼ Request line and header section (382 bytes)

```
GET
http://192.168.100.50:5000/staff/logi
n HTTP/1.1
host: 192.168.100.50:5000
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept:
text/html,application/xhtml+xml,appli
cation/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: http://192.168.100.50:5000/
Upgrade-Insecure-Requests: 1
```

```
Priority: u=0, i
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (189 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:02:16 GMT
Content-Type: text/html; charset=utf-
8
Content-Length: 1931
Vary: Cookie
Connection: close
```

▼ Response body (1931 bytes)

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
content="width=device-width, initial-
scale=1">
  <title>Blood Donation
System</title>
  <link
href="https://cdn.jsdelivr.net/npm/bo
otstrap@5.3.3/dist/css/bootstrap.min.
css" rel="stylesheet">
  <link rel="stylesheet"
href="/static/css/style.css">
</head>
<body>
<nav class="navbar navbar-expand-lg
navbar-dark bg-danger shadow-sm">
  <div class="container">
    <a class="navbar-brand fw-bold"
href="/"> Blood Donation System</a>
```

```
    <button class="navbar-toggler"
type="button" data-bs-
toggle="collapse" data-bs-
target="#nav"><span class="navbar-
toggler-icon"></span></button>
    <div class="collapse navbar-
collapse" id="nav">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a
class="nav-link"
href="/donor/register">Donor
Registration</a></li>

        <li class="nav-item"><a
class="nav-link"
href="/staff/login">Staff Login</a>
</li>

      </ul>
    </div>
  </div>
</nav>
<main>
  <div class="container py-4">




<div class="row justify-content-
center">
  <div class="col-md-5">
    <div class="card shadow-sm
border-0">
      <div class="card-header bg-
white"><h3 class="mb-0">Medical
Staff Login</h3></div>
      <div class="card-body">
        <form method="post">
          <div class="mb-3"><label
class="form-label">Username</label>
<input name="username" class="form-
control" required></div>
          <div class="mb-3"><label
```

```
class="form-label">Password</label>
<input type="password"
name="password" class="form-control"
required></div>
          <button class="btn btn-
danger w-100">Login</button>
        </form>
        <div class="small text-muted
mt-3">Seed account: nurse01 /
Password123</div>
      </div>
    </div>
  </div>
</div>

  </div>
</main>
<script
src="https://cdn.jsdelivr.net/npm/boo
tstrap@5.3.3/dist/js/bootstrap.bundle
.min.js"></script>
</body>
</html>
```

Evidence

```
<form method="post">
```

Solution

Phase: Architecture and Design

Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.

For example, use anti-CSRF packages such as the OWASP CSRFGuard.

Phase: Implementation

Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script.

Phase: Architecture and Design

Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330).

Note that this can be bypassed using XSS.

Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation.

Note that this can be bypassed using XSS.

Use the ESAPI Session Management control.

This control includes a component for CSRF.

Do not use the GET method for any request that triggers a state change.

Phase: Implementation

Check the HTTP Referer header to see if the request originated from an expected page. This could break legitimate functionality, because users or proxies may have disabled sending the Referer for privacy reasons.

Risk=Low, Confidence=High (1)

http://192.168.100.50:5000 (1)

Server Leaks Version Information via "Server" HTTP Response Header Field (1)

▼ GET http://192.168.100.50:5000/static/css/style.css

Alert tags

- OWASP_2021_A05
- POLICY_QA_STD =
- POLICY_PENTEST =
- SYSTEMIC
- OWASP_2017_A06
- WSTG-v42-INFO-02
- CWE-497
- OWASP_2025_A02

Alert description

The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.

Request

▼ Request line and header section (313 bytes)

```
GET
http://192.168.100.50:5000/static/cs
s/style.css HTTP/1.1
host: 192.168.100.50:5000
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/css,*/*;q=0.1
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: http://192.168.100.50:5000/
Priority: u=2
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (367 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:02:14 GMT
Content-Disposition: inline;
filename=style.css
Content-Type: text/css; charset=utf-
8
Content-Length: 763
Last-Modified: Thu, 30 Apr 2026
13:03:00 GMT
Cache-Control: no-cache
ETag: "1777554180.0-763-2936738339"
Date: Fri, 26 Jun 2026 14:02:14 GMT
Connection: close
```

▼ Response body (763 bytes)

```
body { background: #f7f7fb; }
.hero { background: linear-
gradient(135deg, #b00020, #e53935);
}
.blood-drop { font-size: 8rem;
filter: drop-shadow(0 12px 18px
rgba(0,0,0,.25)); }
.stat-card { background: #fff;
border-radius: 1rem; padding:
1.5rem; box-shadow: 0 .5rem 1.5rem
rgba(0,0,0,.07); border-left: 6px
solid #dc3545; }
.stat-card span { color: #6c757d;
display: block; }
.stat-card strong { font-size:
2.5rem; color: #b00020; }
.record-box { border-left: 4px
solid #dc3545; background: #fff5f5;
padding: .75rem; margin-bottom:
.75rem; border-radius: .5rem; }
.update-output { min-height: 240px;
white-space: pre-wrap; background:
```

```
#111827; color: #e5e7eb; padding:
1rem; border-radius: .75rem; }
.card { border-radius: 1rem; }
.btn { border-radius: .6rem; }
```

Evidence `Werkzeug/3.0.3 Python/3.12.3`

Solution Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.

Risk=Low, Confidence=Medium (5)

http://192.168.100.50:5000 (5)

Application Error Disclosure (1)

▼ POST http://192.168.100.50:5000/donor/register

Alert tags

- WSTG-v42-ERRH-02
- WSTG-v42-ERRH-01
- OWASP_2021_A05
- POLICY_QA_STD =
- POLICY_PENTEST =
- CWE-550
- OWASP_2017_A06
- OWASP_2025_A02

Alert description This page contains an error/warning message that may disclose sensitive information like the location of the file that produced the unhandled exception. This information can be used to launch further attacks against the web application. The alert could be a false positive if the error message is found inside a documentation page.

Request

▼ Request line and header section (553 bytes)

```
POST
http://192.168.100.50:5000/donor/reg
ister HTTP/1.1
host: 192.168.100.50:5000
user-agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/141.0.0.0
Safari/537.36
pragma: no-cache
cache-control: no-cache
content-type: application/x-www-
form-urlencoded
referer:
http://192.168.100.50:5000/donor/reg
ister
content-length: 137
Cookie:
session=.eJw9ikEKgCAUBa8iby2RW8_QDUJ
E7FuCKfh1Fd49F9FqGGYe2JAcX8TQ-
wPRJsDde2KGxFbOmMXnoacFZhiJWhJBI_fKN
C9uLgQbD2gl0ZlqdvffV4XxAu2nIi4.aj6Gs
A.x2y2Xh6XTMlB_5l51V5Jzwgj5G0
```

▼ Request body (137 bytes)

```
full_name=ZAP&email=zaproxy%40exampl
e.com&phone=9999999999&ic_number=ZAP
&gender=Female&date_of_birth=2026-
06-26&address=688+Zaproxy+Ridge
```

Response

▼ Status line and header section (195 bytes)

```
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:03:28 GMT
Content-Type: text/html;
charset=utf-8
```

```
Content-Length: 16923
Connection: close
```

▶ Response body (16923 bytes)

Evidence `HTTP/1.1 500 INTERNAL SERVER ERROR`

Solution Review the source code of this page. Implement custom error pages. Consider implementing a mechanism to provide a unique error reference/identifier to the client (browser) while logging the details on the server side and not exposing them to the user.

Cookie No HttpOnly Flag (1)

▼ `POST http://192.168.100.50:5000/staff/login`

Alert tags

- CWE-1004
- OWASP_2021_A05
- POLICY_QA_STD =
- WSTG-v42-SESS-02
- POLICY_PENTEST =
- SYSTEMIC
- OWASP_2017_A06
- OWASP_2025_A02
- POLICY_DEV_STD =

Alert description A cookie has been set without the HttpOnly flag, which means that the cookie can be accessed by JavaScript. If a malicious script can be run on this page then the cookie will be accessible and can be transmitted to another site. If this is a session cookie then session hijacking may be possible.

Request

▼ Request line and header section (499 bytes)

```
POST
http://192.168.100.50:5000/staff/log
in HTTP/1.1
host: 192.168.100.50:5000
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept:
text/html,application/xhtml+xml,appl
ication/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Content-Type: application/x-www-
form-urlencoded
Content-Length: 37
Origin: http://192.168.100.50:5000
Connection: keep-alive
Referer:
http://192.168.100.50:5000/staff/log
in
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

▼ Request body (37 bytes)

```
username=nurse01&password=Password12
3
```

Response

▼ Status line and header section (408 bytes)

```
HTTP/1.1 302 FOUND
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:02:20 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 219
Location: /staff/dashboard
Vary: Cookie
Set-Cookie:
```

```
session=.eJw9ikEKgCAUBa8iby2RW8_QDUJ
E7FuCKfh1Fd49F9FqGGYe2JAcX8TQ-
wPRJsDde2KGxFbOmMXnoacFZhiJWhJBI_fKN
C9uLgQbD2gl0ZlqdvffV4XxAu2nIi4.aj6Gb
A.KZ6mkfmrSp8di6bqglLn6fulP4U;
Path=/
Connection: close
```

▼ Response body (219 bytes)

```
<!doctype html>
<html lang=en>
<title>Redirecting...</title>
<h1>Redirecting...</h1>
<p>You should be redirected
automatically to the target URL: <a
href="/staff/dashboard">/staff/dashb
oard</a>. If not, click the link.
```

Parameter session

Evidence Set-Cookie: session

Solution Ensure that the HttpOnly flag is set for all cookies.

Cookie without SameSite Attribute (1)

▼ POST http://192.168.100.50:5000/staff/login

Alert tags

- OWASP_2021_A01
- POLICY_QA_STD =
- WSTG-v42-SESS-02
- POLICY_PENTEST =
- SYSTEMIC
- CWE-1275
- OWASP_2025_A01
- OWASP_2017_A05
- POLICY_DEV_STD =

Alert description

A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.

Request

▼ Request line and header section (499 bytes)

```
POST
http://192.168.100.50:5000/staff/log
in HTTP/1.1
host: 192.168.100.50:5000
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept:
text/html,application/xhtml+xml,appl
ication/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Content-Type: application/x-www-
form-urlencoded
Content-Length: 37
Origin: http://192.168.100.50:5000
Connection: keep-alive
Referer:
http://192.168.100.50:5000/staff/log
in
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

▼ Request body (37 bytes)

```
username=nurse01&password=Password12
3
```

Response

▼ Status line and header section (408 bytes)

```
HTTP/1.1 302 FOUND
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:02:20 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 219
Location: /staff/dashboard
Vary: Cookie
Set-Cookie:
session=.eJw9ikEKgCAUBa8iby2RW8_QDUJ
E7FuCKfh1Fd49F9FqGGYe2JAcX8TQ-
wPRJsDde2KGxFbOmMXnoacFZhiJWhJBI_fKN
C9uLgQbD2gl0ZlqdvffV4XxAu2nIi4.aj6Gb
A.KZ6mkfmrSp8di6bqglLn6fulP4U;
Path=/
Connection: close
```

▼ Response body (219 bytes)

```
<!doctype html>
<html lang=en>
<title>Redirecting...</title>
<h1>Redirecting...</h1>
<p>You should be redirected
automatically to the target URL: <a
href="/staff/dashboard">/staff/dashb
oard</a>. If not, click the link.
```

Parameter session

Evidence Set-Cookie: session

Solution Ensure that the SameSite attribute is set to either 'lax' or ideally 'strict' for all cookies.

Cross-Domain JavaScript Source File Inclusion (1)

▼ GET http://192.168.100.50:5000/

Alert tags

- OWASP_2025_A08

- POLICY_QA_STD =
- POLICY_PENTEST =
- SYSTEMIC
- OWASP_2021_A08
- CWE-829
- POLICY_DEV_STD =

Alert description

The page includes one or more script files from a third-party domain.

Request

▼ Request line and header section (333 bytes)

```
GET http://192.168.100.50:5000/
HTTP/1.1
host: 192.168.100.50:5000
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept:
text/html,application/xhtml+xml,appl
ication/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (189 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:02:14 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 2563
Vary: Cookie
Connection: close
```

▶ Response body (2563 bytes)

Parameter	`https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js`
Evidence	`<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>`
Solution	Ensure JavaScript source files are loaded from only trusted sources, and the sources can't be controlled by end users of the application.

X-Content-Type-Options Header Missing (1)

▼ `GET http://192.168.100.50:5000/static/css/style.css`

Alert tags	▪ OWASP_2021_A05 ▪ POLICY_QA_STD = ▪ POLICY_PENTEST = ▪ SYSTEMIC ▪ CWE-693 ▪ OWASP_2017_A06 ▪ OWASP_2025_A02
Alert description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.

Other info

This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type.

At "High" threshold this scan rule will not alert on client or server error responses.

Request

▼ Request line and header section (313 bytes)

```
GET
http://192.168.100.50:5000/static/css
/style.css HTTP/1.1
host: 192.168.100.50:5000
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/css,*/*;q=0.1
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: http://192.168.100.50:5000/
Priority: u=2
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (367 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:02:14 GMT
Content-Disposition: inline;
filename=style.css
Content-Type: text/css; charset=utf-8
Content-Length: 763
Last-Modified: Thu, 30 Apr 2026
13:03:00 GMT
```

```
Cache-Control: no-cache
ETag: "1777554180.0-763-2936738339"
Date: Fri, 26 Jun 2026 14:02:14 GMT
Connection: close
```

▼ Response body (763 bytes)

```
body { background: #f7f7fb; }
.hero { background: linear-
gradient(135deg, #b00020, #e53935); }
.blood-drop { font-size: 8rem;
filter: drop-shadow(0 12px 18px
rgba(0,0,0,.25)); }
.stat-card { background: #fff;
border-radius: 1rem; padding:
1.5rem; box-shadow: 0 .5rem 1.5rem
rgba(0,0,0,.07); border-left: 6px
solid #dc3545; }
.stat-card span { color: #6c757d;
display: block; }
.stat-card strong { font-size:
2.5rem; color: #b00020; }
.record-box { border-left: 4px solid
#dc3545; background: #fff5f5;
padding: .75rem; margin-bottom:
.75rem; border-radius: .5rem; }
.update-output { min-height: 240px;
white-space: pre-wrap; background:
#111827; color: #e5e7eb; padding:
1rem; border-radius: .75rem; }
.card { border-radius: 1rem; }
.btn { border-radius: .6rem; }
```

Parameter x-content-type-options

Solution Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages.

If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.

Risk=Informational, Confidence=High (2)

http://192.168.100.50:5000 (2)

Authentication Request Identified (1)

▼ POST http://192.168.100.50:5000/staff/login

Alert tags

Alert description

The given request has been identified as an authentication request. The 'Other Info' field contains a set of key=value lines which identify any relevant fields. If the request is in a context which has an Authentication Method set to "Auto-Detect" then this rule will change the authentication to match the request identified.

Other info

userParam=username

userValue=nurse01

passwordParam=password

referer=http://192.168.100.50:5000/staff/login

Request

▼ Request line and header section (499 bytes)

```
POST
http://192.168.100.50:5000/staff/log
in HTTP/1.1
host: 192.168.100.50:5000
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept:
text/html,application/xhtml+xml,appl
ication/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Content-Type: application/x-www-
form-urlencoded
Content-Length: 37
Origin: http://192.168.100.50:5000
Connection: keep-alive
Referer:
http://192.168.100.50:5000/staff/log
in
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

▼ Request body (37 bytes)

```
username=nurse01&password=Password12
3
```

Response

▼ Status line and header section (408 bytes)

```
HTTP/1.1 302 FOUND
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:02:20 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 219
Location: /staff/dashboard
Vary: Cookie
Set-Cookie:
session=.eJw9ikEKgCAUBa8iby2RW8_QDUJ
E7FuCKfh1Fd49F9FqGGYe2JAcX8TQ-
wPRJsDde2KGxFbOmMXnoacFZhiJWhJBI_fKN
```

```
C9uLgQbD2gl0ZlqdvffV4XxAu2nIi4.aj6Gb
A.KZ6mkfmrSp8di6bqglLn6fulP4U;
Path=/
Connection: close
```

▼ Response body (219 bytes)

```
<!doctype html>
<html lang=en>
<title>Redirecting...</title>
<h1>Redirecting...</h1>
<p>You should be redirected
automatically to the target URL: <a
href="/staff/dashboard">/staff/dashb
oard</a>. If not, click the link.
```

Parameter username

Evidence password

Solution This is an informational alert rather than a vulnerability and so there is nothing to fix.

GET for POST (1)

▼ GET http://192.168.100.50:5000/staff/login

Alert tags

- OWASP_2025_A06
- OWASP_2021_A04
- POLICY_QA_STD =
- POLICY_QA_FULL =
- POLICY_PENTEST =
- WSTG-v42-CONF-06
- POLICY_QA_CICD =
- OWASP_2017_A06
- CWE-16

Alert description A request that was originally observed as a POST was also accepted as a GET. This issue does not represent a security

weakness unto itself, however, it may facilitate simplification of other attacks. For example if the original POST is subject to Cross-Site Scripting (XSS), then this finding may indicate that a simplified (GET based) XSS may also be possible.

Request

▼ Request line and header section (556 bytes)

```
GET
http://192.168.100.50:5000/staff/log
in?password=ZAP&username=ZAP
HTTP/1.1
host: 192.168.100.50:5000
user-agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/141.0.0.0
Safari/537.36
pragma: no-cache
cache-control: no-cache
content-type: application/x-www-
form-urlencoded
referer:
http://192.168.100.50:5000/staff/log
in
Cookie:
session=.eJyNikEKgCAUBa8iby2RW8_QDUJ
C7FuCKfhzJd49F9G61TDMNGw-Wj6JodcGcQ-
Aq3PEDIklHyGJ132NE0yX_zYjUXIkaKRamMb
Ft_V-
Czu0kqhMJdnr67NCfwAwNS9Q.aj6LuQ.N496
QY0n-QjqAMQ1dJpxdc_IcvY
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (321 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:24:57 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 2701
Vary: Cookie
Set-Cookie:
session=eyJyb2xlIjoibnVyc2UiLCJzdGFm
Zl9pZCI6MSwidXNlcm5hbWUiOiJudXJzZTAx
In0.aj6LuQ.dfH_jflzH2Pq2NXLccM7myoDL
c4; Path=/
Connection: close
```

▶ Response body (2701 bytes)

Evidence

```
GET
http://192.168.100.50:5000/staff/log
in?password=ZAP&username=ZAP
HTTP/1.1
```

Solution

Ensure that only POST is accepted where POST is expected.

Risk=Informational, Confidence=Medium (5)

https://cdn.jsdelivr.net (1)

Retrieved from Cache (1)

▼ GET
https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css

Alert tags

- OWASP_2025_A07
- CWE-525
- POLICY_PENTEST =
- OWASP_2021_A07

- SYSTEMIC
- WSTG-v42-ATHN-06
- OWASP_2017_A02

Alert description

The content was retrieved from a shared cache. If the response data is sensitive, personal or user-specific, this may result in sensitive information being leaked. In some cases, this may even result in a user gaining complete control of the session of another user, depending on the configuration of the caching components in use in their environment. This is primarily an issue where caching servers such as "proxy" caches are configured on the local network. This configuration is typically found in corporate or educational environments, for instance.

Request

▼ Request line and header section (410 bytes)

```
GET
https://cdn.jsdelivr.net/npm/bootstra
p@5.3.3/dist/css/bootstrap.min.css
HTTP/1.1
host: cdn.jsdelivr.net
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/css,*/*;q=0.1
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: http://192.168.100.50:5000/
Sec-Fetch-Dest: style
Sec-Fetch-Mode: no-cors
Sec-Fetch-Site: cross-site
Priority: u=2
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (768 bytes)

```
HTTP/1.1 200 OK
Connection: keep-alive
Content-Length: 232803
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: *
Timing-Allow-Origin: *
Cache-Control: public, max-age=31536000, s-maxage=31536000, immutable
Cross-Origin-Resource-Policy: cross-origin
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
Content-Type: text/css; charset=utf-8
X-JSD-Version: 5.3.3
X-JSD-Version-Type: version
ETag: W/"38d63-xawd7pYctZoEUlbsID9p4xeHL3w"
Accept-Ranges: bytes
Age: 2703802
Date: Fri, 26 Jun 2026 14:02:15 GMT
X-Served-By: cache-fra-etou8220150-FRA, cache-sin-wsss1830094-SIN
X-Cache: HIT, HIT
Vary: Accept-Encoding
alt-svc: h3=":443";ma=86400,h3-29=":443";ma=86400,h3-27=":443";ma=86400
```

▶ Response body (232803 bytes)

Evidence

HIT

Solution

Validate that the response does not contain sensitive, personal or user-specific information. If it does, consider

the use of the following HTTP response headers, to limit, or prevent the content being stored and retrieved from the cache by another user:

Cache-Control: no-cache, no-store, must-revalidate, private

Pragma: no-cache

Expires: 0

This configuration directs both HTTP 1.0 and HTTP 1.1 compliant caching servers to not store the response, and to not retrieve the response (without validation) from the cache, in response to a similar request.

http://192.168.100.50:5000 (4)

Information Disclosure - Suspicious Comments (1)

▼ POST http://192.168.100.50:5000/donor/register

Alert tags

- OWASP_2021_A01
- WSTG-v42-INFO-05
- OWASP_2017_A03
- OWASP_2025_A01
- POLICY_PENTEST =
- CWE-615

Alert description

The response appears to contain suspicious comments which may help an attacker.

Other info

The following pattern was used: \bQUERY\b and was detected in likely comment: "<!--

Traceback (most recent call last):

File "/opt/blood-donation-system/venv/lib/python3.12/site-packages/flask/app.py", lin", see evidence field for the suspicious comment/snippet.

Request

▼ Request line and header section (553 bytes)

```
POST
http://192.168.100.50:5000/donor/reg
ister HTTP/1.1
host: 192.168.100.50:5000
user-agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/141.0.0.0
Safari/537.36
pragma: no-cache
cache-control: no-cache
content-type: application/x-www-
form-urlencoded
referer:
http://192.168.100.50:5000/donor/reg
ister
content-length: 137
Cookie:
session=.eJw9ikEKgCAUBa8iby2RW8_QDUJ
E7FuCKfh1Fd49F9FqGGYe2JAcX8TQ-
wPRJsDde2KGxFbOmMXnoacFZhiJWhJBI_fKN
C9uLgQbD2gl0ZlqdvffV4XxAu2nIi4.aj6Gs
A.x2y2Xh6XTMlB_5l51V5Jzwgj5G0
```

▼ Request body (137 bytes)

```
full_name=ZAP&email=zaproxy%40exampl
e.com&phone=9999999999&ic_number=ZAP
&gender=Female&date_of_birth=2026-
06-26&address=688+Zaproxy+Ridge
```

Response

▼ Status line and header section (195 bytes)

```
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:03:28 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 16923
Connection: close
```

▶ Response body (16923 bytes)

Evidence

```
urn super().execute(query, vars)
            ^
```

Solution

Remove all comments that return information that may help an attacker and fix any underlying problems they refer to.

Modern Web Application (1)

▼ `POST http://192.168.100.50:5000/donor/register`

Alert tags

- OWASP_2021_A05
- POLICY_QA_STD =
- POLICY_PENTEST =
- SYSTEMIC
- OWASP_2017_A06
- OWASP_2025_A02
- POLICY_DEV_STD =

Alert description

The application appears to be a modern web application. If you need to explore it automatically then the Ajax Spider may well be more effective than the standard one.

Other info

No links have been found while there are scripts, which is an indication that this is a modern web application.

Request

▼ Request line and header section (553 bytes)

```
POST
http://192.168.100.50:5000/donor/reg
ister HTTP/1.1
host: 192.168.100.50:5000
user-agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/141.0.0.0
Safari/537.36
pragma: no-cache
cache-control: no-cache
content-type: application/x-www-
form-urlencoded
referer:
http://192.168.100.50:5000/donor/reg
ister
content-length: 137
Cookie:
session=.eJw9ikEKgCAUBa8iby2RW8_QDUJ
E7FuCKfh1Fd49F9FqGGYe2JAcX8TQ-
wPRJsDde2KGxFbOmMXnoacFZhiJWhJBI_fKN
C9uLgQbD2gl0ZlqdvffV4XxAu2nIi4.aj6Gs
A.x2y2Xh6XTMlB_5l51V5Jzwgj5G0
```

▼ Request body (137 bytes)

```
full_name=ZAP&email=zaproxy%40exampl
e.com&phone=9999999999&ic_number=ZAP
&gender=Female&date_of_birth=2026-
06-26&address=688+Zaproxy+Ridge
```

Response

▼ Status line and header section (195 bytes)

```
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:03:28 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 16923
Connection: close
```

▶ Response body (16923 bytes)

Evidence

```
<script src="?
__debugger__=yes&amp;cmd=resource&am
p;f=debugger.js"></script>
```

Solution This is an informational alert and so no changes are required.

Session Management Response Identified (1)

▼ `GET http://192.168.100.50:5000/staff/dashboard`

Alert tags

Alert description The given response has been identified as containing a session management token. The 'Other Info' field contains a set of header tokens that can be used in the Header Based Session Management Method. If the request is in a context which has a Session Management Method set to "Auto-Detect" then this rule will change the session management to use the tokens identified.

Other info cookie:session

Request ▼ Request line and header section (574 bytes)

```
GET
http://192.168.100.50:5000/staff/das
hboard HTTP/1.1
host: 192.168.100.50:5000
User-Agent: Mozilla/5.0 (X11; Linux
x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept:
text/html,application/xhtml+xml,appl
ication/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Referer:
http://192.168.100.50:5000/staff/log
in
Connection: keep-alive
Cookie:
session=.eJw9ikEKgCAUBa8iby2RW8_QDUJ
E7FuCKfh1Fd49F9FqGGYe2JAcX8TQ-
wPRJsDde2KGxFbOmMXnoacFZhiJWhJBI_fKN
C9uLgQbD2gl0ZlqdvffV4XxAu2nIi4.aj6Gb
A.KZ6mkfmrSp8di6bqglLn6fulP4U
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (321 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:02:20 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 2731
Vary: Cookie
Set-Cookie:
session=eyJyb2xlIjoibnVyc2UiLCJzdGFm
Zl9pZCI6MSwidXNlcm5hbWUiOiJudXJzZTAx
In0.aj6GbA.OSvPK5Yp5G7ZoGMZuLLz1w3HP
to; Path=/
```

```
Connection: close
```

▶ Response body (2731 bytes)

Parameter	session
Evidence	session
Solution	This is an informational alert rather than a vulnerability and so there is nothing to fix.

User Agent Fuzzer (1)

▼ GET http://192.168.100.50:5000/

Alert tags	▪ CUSTOM_PAYLOADS = ▪ POLICY_PENTEST = ▪ SYSTEMIC
Alert description	Check for differences in response based on fuzzed User Agent (eg. mobile sites, access as a Search Engine Crawler). Compares the response statuscode and the hashcode of the response body with the original response.
Request	▼ Request line and header section (298 bytes)

```
GET http://192.168.100.50:5000/
HTTP/1.1
host: 192.168.100.50:5000
user-agent: Mozilla/4.0
(compatible; MSIE 7.0; Windows NT
6.0)
pragma: no-cache
cache-control: no-cache
Cookie:
session=eyJyb2xlIjoibnVyc2UiLCJzdGFm
```

Zl9pZCI6MSwidXNlcm5hbWUiOiJudXJzZTAx
In0.aj6Lug.ksEDPm0uLdM21UG-
78yOKHy2Ioc

▼ Request body (0 bytes)

Response

▼ Status line and header section (189 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:24:58 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 2863
Vary: Cookie
Connection: close
```

▶ Response body (2863 bytes)

Parameter Header User-Agent

Attack Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.0)

Risk=Informational, Confidence=Low (1)

http://192.168.100.50:5000 (1)

User Controllable HTML Element Attribute (Potential XSS) (1)

▼ GET http://192.168.100.50:5000/donor/donation-pass?donor_id=1&ref=BDP-1

Alert tags

- OWASP_2025_A05

- OWASP_2017_A01
- OWASP_2021_A03
- CWE-20
- POLICY_PENTEST =

Alert description

This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.

Other info

User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL:

http://192.168.100.50:5000/donation-pass?donor_id=1&ref=BDP-1

appears to include user input in:

a(n) [meta] tag [content] attribute

The user input found was:

donor_id=1

The user-controlled value was:

width=device-width, initial-scale=1

Request

▼ Request line and header section (519 bytes)

```
GET
http://192.168.100.50:5000/donor/donation-pass?donor_id=1&ref=BDP-1
HTTP/1.1
host: 192.168.100.50:5000
```

```
user-agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/141.0.0.0
Safari/537.36
pragma: no-cache
cache-control: no-cache
referer:
http://192.168.100.50:5000/staff/rep
orts/operational
Cookie:
session=.eJw9ikEKgCAUBa8iby2RW8_QDUJ
E7FuCKfh1Fd49F9FqGGYe2JAcX8TQ-
wPRJsDde2KGxFbOmMXnoacFZhiJWhJBI_fKN
C9uLgQbD2gl0ZlqdvffV4XxAu2nIi4.aj6Gs
A.x2y2Xh6XTMlB_5l51V5Jzwgj5G0
```

▼ Request body (0 bytes)

Response

▼ Status line and header section (321 bytes)

```
HTTP/1.1 200 OK
Server: Werkzeug/3.0.3 Python/3.12.3
Date: Fri, 26 Jun 2026 14:03:28 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 3659
Vary: Cookie
Set-Cookie:
session=eyJyb2xlIjoibnVyc2UiLCJzdGFm
Zl9pZCI6MSwidXNlcm5hbWUiOiJudXJzZTAx
In0.aj6GsA.VZFKCGsisNmLicQQtSSkBfRP-
YA; Path=/
Connection: close
```

▶ Response body (3659 bytes)

Parameter donor_id

Solution	Validate all input and sanitize output it before writing to any HTML attributes.

Appendix

Alert Types

This section contains additional information on the types of alerts in the report.

Cross Site Scripting (Persistent)

Source	raised by an active scanner (Cross Site Scripting (Persistent))
CWE ID	79
WASC ID	8
Reference	▪ https://owasp.org/www-community/attacks/xss/ ▪ https://cwe.mitre.org/data/definitions/79.html

Cross Site Scripting (Reflected)

Source	raised by an active scanner (Cross Site Scripting (Reflected))
CWE ID	79
WASC ID	8
Reference	▪ https://owasp.org/www-community/attacks/xss/

- https://cwe.mitre.org/data/definitions/79.html

Path Traversal

Source	raised by an active scanner (Path Traversal)
CWE ID	22
WASC ID	33
Reference	• https://owasp.org/www-community/attacks/Path_Traversal • https://cwe.mitre.org/data/definitions/22.html

SQL Injection

Source	raised by an active scanner (SQL Injection)
CWE ID	89
WASC ID	19
Reference	• https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Absence of Anti-CSRF Tokens

Source	raised by a passive scanner (Absence of Anti-CSRF Tokens)
CWE ID	352
WASC ID	9
Reference	• https://cheatsheetseries.owasp.org/cheatsheet

s/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

- https://cwe.mitre.org/data/definitions/352.html

Buffer Overflow

Source	raised by an active scanner (Buffer Overflow)
CWE ID	120
WASC ID	7
Reference	- https://owasp.org/www-community/attacks/Buffer_overflow_attack

Content Security Policy (CSP) Header Not Set

Source	raised by a passive scanner (Content Security Policy (CSP) Header Not Set)
CWE ID	693
WASC ID	15
Reference	- https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP - https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html - https://www.w3.org/TR/CSP/ - https://w3c.github.io/webappsec-csp/ - https://web.dev/articles/csp

- https://caniuse.com/#feat=contentsecuritypolicy
- https://content-security-policy.com/

Cross-Domain Misconfiguration

Source	raised by a passive scanner (Cross-Domain Misconfiguration)
CWE ID	264
WASC ID	14
Reference	• https://vulncat.fortify.com/en/detail?category=HTML5&subcategory=Overly%20Permissive%20CORS%20Policy

Format String Error

Source	raised by an active scanner (Format String Error)
CWE ID	134
WASC ID	6
Reference	• https://owasp.org/www-community/attacks/Format_string_attack

Missing Anti-clickjacking Header

Source	raised by a passive scanner (Anti-clickjacking Header)
CWE ID	1021
WASC ID	15

Reference	▪ https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options

Sub Resource Integrity Attribute Missing

Source	raised by a passive scanner (Sub Resource Integrity Attribute Missing)
CWE ID	345
WASC ID	15
Reference	▪ https://developer.mozilla.org/en-US/docs/Web/Security/Subresource_Integrity

Application Error Disclosure

Source	raised by a passive scanner (Application Error Disclosure)
CWE ID	550
WASC ID	13

Cookie No HttpOnly Flag

Source	raised by a passive scanner (Cookie No HttpOnly Flag)
CWE ID	1004
WASC ID	13
Reference	▪ https://owasp.org/www-community/HttpOnly

Cookie without SameSite Attribute

Source	raised by a passive scanner (Cookie without SameSite Attribute)
CWE ID	1275
WASC ID	13
Reference	▪ https://datatracker.ietf.org/doc/html/draft-ietf-httpbis-cookie-same-site

Cross-Domain JavaScript Source File Inclusion

Source	raised by a passive scanner (Cross-Domain JavaScript Source File Inclusion)
CWE ID	829
WASC ID	15

Server Leaks Version Information via "Server" HTTP Response Header Field

Source	raised by a passive scanner (HTTP Server Response Header)
CWE ID	497
WASC ID	13
Reference	▪ https://httpd.apache.org/docs/current/mod/core.html#servertokens ▪ https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10) ▪ https://www.troyhunt.com/shhh-dont-let-your-response-headers/

X-Content-Type-Options Header Missing

Source	raised by a passive scanner (X-Content-Type-Options Header Missing)
CWE ID	693
WASC ID	15
Reference	▪ https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) ▪ https://owasp.org/www-community/Security_Headers

Authentication Request Identified

Source	raised by a passive scanner (Authentication Request Identified)
Reference	▪ https://www.zaproxy.org/docs/desktop/addons/authentication-helper/auth-req-id/

GET for POST

Source	raised by an active scanner (GET for POST)
CWE ID	16
WASC ID	20

Information Disclosure - Suspicious Comments

Source	raised by a passive scanner (Information Disclosure - Suspicious Comments)
CWE ID	615

WASC ID	13

Modern Web Application

Source	raised by a passive scanner (Modern Web Application)

Retrieved from Cache

Source	raised by a passive scanner (Retrieved from Cache)
CWE ID	525
Reference	▪ https://datatracker.ietf.org/doc/html/rfc7234 ▪ https://datatracker.ietf.org/doc/html/rfc7231 ▪ https://www.rfc-editor.org/rfc/rfc9110.html

Session Management Response Identified

Source	raised by a passive scanner (Session Management Response Identified)
Reference	▪ https://www.zaproxy.org/docs/desktop/addons/authentication-helper/session-mgmt-id/

User Agent Fuzzer

Source	raised by an active scanner (User Agent Fuzzer)
Reference	▪ https://owasp.org/wstg

User Controllable HTML Element Attribute (Potential XSS)

Source	raised by a passive scanner (User Controllable HTML Element Attribute (Potential XSS))
CWE ID	20
WASC ID	20
Reference	▪ https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html